

epochs, which is the difference between a fine-tune that might beat stock and one that probably does.

THE ONE THING THAT MUST HAPPEN THURSDAY

Block T3, around hour 2:30 — **launch the overnight fine-tune**. Everything else on Thursday can slip by an hour without consequence. If that job is not running when you go to sleep, Friday loses its head start and the whole plan drops back to a compromise.

THE GOVERNING DECISION

Train, fit, or prompt

The footage arrives at the event. Anything trained now on Avenue or UCF-Crime learns a different city, altitude, camera and lighting — and **"normal" is precisely what will not transfer**. What *does* transfer is viewpoint and scale. So every capability in this build is sorted into one of three buckets, and knowing which is which is the strongest thing you can say in Q&A.

GRADIENT DESCENT

Train — overnight

Domain-general. Survives the change of city.

- `yolo11s` fine-tuned on VisDrone — aerial, small, top-down
- 40% synthetic night copies, so low light isn't a surprise

Launched T3 · collected F0

ARITHMETIC

Fit — at the event

Scene-specific. Cheap, because nothing is learned.

- Per-class speed mean and std
- Density / congestion baseline
- Normal-bank embeddings from quiet frames
- Zone polygons, drawn by hand
- Thresholds, swept on whatever labels exist

`fit.py` · under 5 minutes

TEXT

Prompt — never train

Your real tuning surface on the day.

- The open-vocabulary class list
- The VLM judge prompt and its JSON schema

Seconds. Iterate live.

"Everything trainable that is domain-general was trained before I arrived. Everything scene-specific is fitted on your footage in five minutes, because it is arithmetic rather than gradient descent. And the class list is text, so I can change what counts as an anomaly while you watch."

DO NOT FINE-TUNE THE VLM — EVEN WITH 22 HOURS

You know LoRA and PEFT, which makes this the most tempting wrong move available. Without their footage and without labels it is hours of GPU for no measurable gain, and it competes with the overnight detector run for the same device. **The brief asks an architectural question, not a leaderboard one.** Prompt the judge; don't retrain it.

THE STACK · ONE MODEL PER STAGE, DECIDED

What we are actually using

It looks like a long list, so here it is sorted by what actually costs you something. **Two models do the work.** Three more are level-gated add-ons. Four rows on the old version of this table were not models at all.

Core — load these first, they answer the brief on their own

MODEL	SIZE	DOES WHAT NOTHING ELSE CAN	FALLBACK
YOLO-World-L CORE	~90 MB	Detection <i>and</i> open-vocabulary — text is the class list. Nothing else can name a thing you never trained for, which is what "anomaly" means	Grounding DINO Swin-T
Qwen2.5-VL-3B-Instruct CORE	~2.5 GB 4-bit	The judgement, and the sentence explaining it. Literally the "small VLM" in the problem statement	SmolVLM2-2.2B
Level 1 is answerable with these two alone			~2.6 GB total

Level-gated — add each when its level arrives

MODEL	SIZE	WHAT IT BUYS	ADD AT
VisDrone-pretrained YOLO + our night fine-tune OURS	~19 MB	A fast closed-set pass so YOLO-World only fires on candidates. A <i>speed</i> optimisation, not a capability — it is what makes the FPS number real	F3
SigLIP ViT-B-16 (webli) DUAL USE	~800 MB	One index serving the normal-bank anomaly score <i>and</i> text retrieval over the footage	F2
OSNet-x1.0 (torchreid)	~9 MB	Re-ID across occlusion gaps. Only if needed — Ultralytics' BoT-SORT can reuse the detector's own features, so check the tracker YAML first	F1
SAM 2.1 hiera-small	~180 MB	Masks that propagate through video themselves. A placing direction last year — strictly optional	F7
Everything on disk, with Qwen at 4-bit ≈ 3.5 GB — space is not the constraint here			

Not models — no weights, no download, no disk

ROW	WHAT IT ACTUALLY IS
SAHI tiling, 640 @ 0.2	A slicing technique wrapping the detector you already loaded — runs it on tiles instead of one resize. A 15 px top-down person survives tiling and dies in a 640 resize
BoT-SORT	A tracking algorithm: Kalman filter plus Hungarian matching over boxes. Config, not weights
CLAHE + separate night conf	An OpenCV histogram function, about five lines. Recovers contrast before inference instead of hoping the detector generalises
TensorRT export	A compiler that re-exports the detector you already have. You know ONNX, so it is a one-liner

WHY IT CANNOT BE ONE MODEL

Run *only* Qwen over sampled frames and you get one to three seconds per frame — hours for ten minutes of footage. That is exactly the question they asked, and the one-model answer is **no**. A cheap gate in front of the VLM is not architectural decoration; it is the finding. **Two is the floor because the brief's question has a floor.**

START FROM SOMEONE ELSE'S AERIAL PRETRAINING, NOT FROM COCO

Before launching block T3, spend ten minutes hunting a **VisDrone-pretrained YOLO checkpoint** on HF Hub or GitHub. Someone has already done the top-down, small-object, drone-altitude adaptation. Starting there and spending your overnight GPU only on the *night* gap is a two-stage fine-tune that beats one stage from COCO, for less compute. Swap the path in `train_aerial.py` and nothing else changes. Verify it really was trained on VisDrone, check the licence, and A/B it in F0 like everything else — a random community checkpoint can be worse than stock.

ON YOLO VERSIONS

Run `pip install -U ultralytics` and use whatever the newest checkpoint is — the API is identical, so it is a one-string change. **But do not spend time on it.** A version bump buys a couple of mAP points; `imgsz=1280` plus SAHI buys far more on aerial footage. YOLO-World is built on a YOLOv8 backbone regardless, so a newer YOLO does not give you a newer open-vocabulary model. *(This plan was written against a May 2026 knowledge cutoff, so check what has shipped since.)*

NVIDIA'S SHELF, AND ITS CATCH

You were right that the market has most of this. **ReIdentificationNet** is the highest value-per-hour item on NGC/TAO — person re-ID pretrained on far more surveillance footage than you can touch in two days — with **PeopleNet** (elevated-camera person/face/bag) close behind, and **DeepStream** and the **VSS blueprint** worth reading for architecture rather than adopting as runtime. The catch is format: TAO checkpoints usually want the TAO toolkit or DeepStream to consume, so they sit off the critical path as upgrades *after* a level is cleared. These names are from memory — the search budget ran out mid-task — so

confirm them on NGC before relying on them.

HOW IT FITS TOGETHER

The pipeline, module by module

Every stage reads a file and emits a file, so any stage can be run, inspected and debugged alone. That property is what makes a bad hour survivable.

MODULE	READS	EMITS	CONSUMED BY
fit.py	their video, config.yaml	out/scene_fit.json , out/normal_bank.npy	events, fusion, retrieval
pipeline/tracks.py	video, detector weights	{track_id: Tracklet} — dwell, speed, boxes	events.py , re-ID
pipeline/events.py	tracklets + fitted baselines	CandidateEvent with the facts that justified it	vlm_judge.py
pipeline/openvocab.py T8	candidate windows, text prompt list	extra boxes for classes COCO lacks	vlm_judge.py
pipeline/vlm_judge.py	candidate + 6-frame strip	{anomalous, score, label, why}	fuse.py
pipeline/fuse.py	geo score + VLM verdict	alerts, after hysteresis and cooldown	demo UI, evaluate.py
pipeline/reid.py F1	tracklet crops	appearance embeddings + identity links	events.py , demo UI
pipeline/retrieve.py F2	normal_bank.npy , a text query	ranked frame indices → clip	demo UI
pipeline/evaluate.py	alerts + labels	AUC, P/R/F1, FP/hour, latency	your slides
run.py	everything above	out/alerts.json + a console table	you, at 09:20 on Saturday

Modules marked with a block tag do not exist yet

They are built in the block named

And the cost argument the shape encodes

Live stream 10 min @ 30 fps	18,000 frames
Frame sampling target_fps = 3	1,800 frames
yolo11s-aerial + BoT-SORT closed-set, always on	~40 tracklets
Geometry vs fitted baselines dwell · zone · speed-z · density	~8 candidates
YOLO-World text prompts, candidates only	8 windows
Qwen2.5-VL-3B judges 6-frame strip, JSON out	8 calls
Fuse + suppress hysteresis · cooldown	~3 alerts
Bars are log-scaled. 8 VLM calls instead of 18,000 – roughly 2,000× fewer, sampling 48 images, or 0.27% of the footage. A parallel SigLIP pass over the same frames is both the normal-bank score and the retrieval index.	

THURSDAY · 10 HOURS

Get the spine running, then launch the GPU

Today's job is a pipeline that runs end to end on *any* footage, plus a training job in flight before bed. Depth comes Friday.

#	CLOCK	BUILD	DELIVERABLE BEFORE YOU MOVE ON

T0	2:00–2:30 PM	GPU + env decision	<code>nvidia-smi</code> answered, or a Kaggle notebook open. VisDrone, Avenue, YOLO-World and Qwen downloads all started. CHECKPOINT You can state your VRAM number and which training preset it allows, without looking it up again.
T1	2:30–3:45	PyTorch narrow list + scaffold runs	<code>run.py</code> executes on any clip. Read the PyTorch rows while pip installs. RESOURCE CUDA semantics · inference_mode · Ultralytics predict args CHECKPOINT Explain out loud, no notes: what <code>torch.inference_mode()</code> does, and why forgetting it OOMs a GPU that has plenty of room.
T2	3:45–4:30	Night dataset	<code>make_night.py</code> done, and you have <i>looked at</i> ten darkened images. Too dark is a real failure mode. CHECKPOINT Open three night images beside their originals. If you cannot see the objects, neither can the detector — lower the gamma range and rerun.
T3	4:30–5:00	Launch the fine-tune ← cannot slip	Training running, epoch 1 loss printed, wall-clock extrapolated to finish before you wake. CHECKPOINT Epoch-1 time × 60 × 1.15 lands before your alarm. If not, kill it and relaunch on <code>--preset kaggle</code> or <code>laptop</code> right now.
–	5:00–5:30	Break	Stand up. The GPU is working; you don't have to.
T4	5:30–6:30	Tracking	BoT-SORT tracklets carrying dwell and speed; ID switches counted by eye over 30 s. RESOURCE Ultralytics track · ByteTrack abstract + Fig. 2 CHECKPOINT Explain in two sentences how ByteTrack keeps IDs across frames — specifically what it does with the <i>low-confidence</i> boxes everyone else throws away.
T5	6:30–7:45	Event layer + <code>fit.py</code> ← most marks	Loiter, zone, speed-z and density firing — and <code>fit.py</code> emitting baselines in under two minutes. CHECKPOINT

			Explain why the speed baseline is measured per-video rather than hardcoded, and what <code>fit.py</code> does when it sees fewer than twenty samples.
—	7:45–8:15	Dinner	Away from the screen. Check the training loss on your way back, not during.
T6	8:15–9:30	The judge	Qwen2.5-VL-3B returning parseable JSON on ~20 candidates, ms per call measured. RESOURCE Qwen2.5-VL cookbooks · search YouTube: Umar Jamil "Vision Language Models" — code-along, explains encoder → projection → decoder clearly CHECKPOINT Describe the vision-encoder → projection-layer → language-model path in your own words. Then run the same clip at 1 frame, 6 frames, and 6 frames plus track history, and say which verdict changed and why.
T7	9:30–10:30	Fusion + first metrics	Hysteresis and cooldown working; a full metrics table on CUHK Avenue. CHECKPOINT State your alert count before and after suppression as one sentence — that line goes on a slide unchanged.
T8	10:30–11:30	Open-vocabulary	YOLO-World detecting something COCO has no class for, on candidate windows only. RESOURCE YOLO-World docs · CLIP paper (abstract + intro only) CHECKPOINT Explain why contrastive image-text training is what makes zero-shot detection possible at all — and therefore why an anomaly, which has no class label, needs it.
T9	11:30–12:15 AM	Night path	Detection counts four ways: day/night × before/after CLAHE, on stock weights. RESOURCE OpenCV CLAHE CHECKPOINT Four numbers written down. You should be able to say which of the four is worst and what you would change first.
T10	12:15–12:45	Download all, load all	Every checkpoint fetched <i>and loaded once</i>. Training confirmed progressing. Broken things written down for tomorrow. CHECKPOINT

Each model imported and instantiated in a throwaway script.
"Downloaded" is not "loads".

-	12:45-7:15 AM	Sleep — non-negotiable	The GPU works all night. You will demo worse exhausted than under-studied.
---	---------------	------------------------	--

If Thursday runs short, drop T8 and T9

Never drop T3

```
# T0 - the answer that decides everything else
nvidia-smi

# T2 - VisDrone auto-downloads on first train call; then darken 40% of it
python train/make_night.py --images datasets/VisDrone/images/train \
                           --labels datasets/VisDrone/labels/train --fraction 0.4

# T3 - launch and walk away. yolo11s @1280, 60 epochs, no freeze.
#      BETTER: if you found a VisDrone-pretrained checkpoint, start from it.
#      Edit PRESETS in train_aerial.py: "yolo11s.pt" -> "visdrone_pretrained.pt"
python train/train_aerial.py --preset overnight
#  laptop -> yolo11n @960, 25 ep, freeze=10    ~3-4 h
#  kaggle  -> yolo11s @1024, 40 ep, no freeze   ~5-6 h (free P100/T4)
#  fast    -> yolo11n @768, 12 ep, freeze=10   ~1-1.5 h

# T5 - prove the fit path on footage the pipeline has never seen
python fit.py --video data/avenue_01.avi --fit-seconds 120

# T7 - first real numbers
python run.py --video data/avenue_01.avi --set vlm.backend=qwen
```

CHECK THE EXTRAPOLATION BEFORE YOU SLEEP

Watch epoch one, multiply by 60, and add 15%. If that lands past when you plan to wake, kill it and relaunch on `--preset kaggle` or `laptop`. A job you have to stop at epoch 30 gives you a usable `last.pt`, but a job still running when you need the GPU for Friday's work is a straight loss.

OVERNIGHT · ~9 HOURS, UNATTENDED

What runs while you sleep

THE GPU Fine-tuning the aerial detector yolo11s · 1280 px · 60 epochs	THE NETWORK Everything else, downloading Started T0, finished by morning
data VisDrone train, plus 40% synthetic night twins with labels copied verbatim	data CUHK Avenue (frame labels — your AUC), a VisDrone video sample
aug hsv_v=0.65 brightness jitter, mosaic on with close_mosaic=8 , mild rotate and scale	models YOLO-World-L, Qwen2.5-VL-3B, SmolVLM2, SigLIP, OSNet, SAM 2.1
out weights/aerial_night/weights/best.pt — and last.pt , if you have to stop it early	
risk It may lose to stock. That is a normal outcome, and block F0 exists to find out	why Seventy people on one venue wifi. A checkpoint you fetch on Saturday is a checkpoint you may not get

FRIDAY · 12 HOURS

Collect the weights, then build the levels

Friday's job is everything the brief lists that Thursday didn't reach — identity, retrieval, forensics, real-time — plus the demo surface and two dry runs. Rehearsal is not optional; it is blocks F9 and F10.

#	CLOCK	BUILD	DELIVERABLE BEFORE YOU MOVE ON
F0	7:30–8:15 AM	Breakfast + collect the weights	mAP table: stock vs fine-tuned, day and night. Keep stock if it wins, and say so on the slide. CHECKPOINT Four mAP numbers in a table. If the fine-tune lost, you can say in one sentence why that is a legitimate result rather than a wasted night.
F1	8:15–9:45	Re-identification	Appearance embeddings per tracklet; one identity matched across an occlusion gap. RESOURCE torchreid / OSNet · NVIDIA TAO zoo (verify names) CHECKPOINT

			Check the BoT-SORT YAML first — if its own features suffice, you have saved a model. State both failure directions of the cosine threshold: strangers merging vs one person becoming three.
F2	9:45–11:15	Retrieval	A typed query — "person carrying a bag near the gate" — returns the right clip. RESOURCE open_clip · search: "huggingface smol-vision" · search: WinCLIP, AnomalyCLIP — zero-shot anomaly detection with CLIP, the same idea as your normal bank CHECKPOINT Explain why one embedding index serves both retrieval and anomaly scoring — and why that makes the normal bank zero-shot rather than trained.
F3	11:15–12:15 PM	Real-time loop	Sustained FPS on a stream, TensorRT export done, both numbers written down. RESOURCE TensorRT export CHECKPOINT One number you would say to a judge: "X FPS sustained, end to end, on this machine." Not a benchmark — your pipeline.
—	12:15–1:00	Lunch	Away from the screen.
F4	1:00–2:30	Demo surface ← judges see this	Timeline of alerts, click to play the evidence clip, the one-sentence "why" beside each. CHECKPOINT Hand the laptop to someone who has not seen it. If they cannot tell what the system found and why within thirty seconds, rebuild the screen, not the model.
F5	2:30–3:45	Full evaluation sweep	Day AUC and night AUC separately, threshold sweep, FP per hour, p50/p95 latency, time-to-detection. RESOURCE sklearn metrics · search: NVIDIA AI City Challenge Track 4 — for how a live-monitoring task defines its metrics CHECKPOINT Explain why <i>time-to-detection</i> matters more than raw accuracy for a live system — an alert that arrives four minutes late is a report, not an alarm.
F6	3:45–4:45	Exercise	A VLM summary across several events in one long

F6	3:45-4:45	forensic reasoning	Ten-Hour Flight Plan A VLM summary across several events in one long window — the level-3 story. RESOURCE search: LAVAD, "training-free video anomaly detection" — the method figure CHECKPOINT One paragraph the system wrote about a five-minute window, citing timestamps. Read it aloud and check every claim against the footage.
—	4:45-5:15	Break	Last one before the rehearsals.
F7	5:15-6:00	Optional upgrade	Pick one: SAM 2 masking, map grounding, or congestion analysis. All three placed last year. CHECKPOINT If it is not working by 6:00, delete it. A half-wired upgrade is worse than none.
F8	6:00-7:00	Harden	Named presets — <code>day</code>, <code>night</code>, <code>fast</code>, <code>accurate</code> — each one command. CHECKPOINT Every preset runs from a cold shell with one line. Saturday morning types commands, it does not edit code.
F9	7:00-8:00	Dry run one	An unseen clip, start to finish, nothing touched. Fix whatever broke. CHECKPOINT Time it. That number is your honest estimate for Saturday's first thirty minutes.
F10	8:00-9:00	Dry run two, timed	An unseen <i>night</i> clip in twenty minutes, three parameters changed live while narrating. CHECKPOINT You changed <code>conf</code>, a prompt, and a threshold while talking, and explained each one as you typed. This is the skill the arena actually tests.
F11	9:00-10:00	Five slides, then stop	Problem, architecture, the numbers, the honest limitations, the one sentence. CHECKPOINT Say the two-minute pitch out loud, once, uninterrupted, timed. Then close the laptop and pack.
—	10:30-6:00	Sleep, also	Nothing you build after 10 PM survives contact with a

9/3/26, 11:04 PM

10:30-11:00 AM	Sleep — also non-negotiable	Ten-Hour Flight Plan Nothing you build after 10 PM survives contact with a judge.
If Friday runs short, drop F7 and shorten F6		Never drop F9 and F10 — an unrehearsed demo fails in public

```
# F0 - A/B before you trust the thing you trained. Night clip specifically.
yolo val model=weights/aerial_night/weights/best.pt data=VisDrone.yaml imgsz=1280
yolo val model=yolo11s.pt data=VisDrone.yaml imgsz=1280

# F3 - the FPS number for your slide
yolo export model=weights/aerial_night/weights/best.pt format=engine half=True

# F8 - what Saturday morning actually types
python fit.py --video theirs.mp4 # ~5 min, scene baselines
python run.py --video theirs.mp4 --preset night # alerts + metrics
```

SATURDAY · THE ARENA

The first thirty minutes

If Thursday and Friday went to plan, level one is cleared before the room has finished settling. Nothing here trains anything, and nothing here is new material.

WALL CLOCK	BEFORE YOU LEAVE	CHECKPOINT
6:30–7:30 AM	Rapid-fire recall — out loud, no notes Go topic by topic from memory: PyTorch device and memory → why the cascade exists → ByteTrack's low-confidence trick → vision-encoder → projection → decoder → why contrastive training enables zero-shot → what <code>fit.py</code> measures → your metrics and what each one answers.	Anything you blank on becomes a 20-minute review — not a new topic . No new material on the day.
7:30–8:15 AM	Final dry run, wifi off Full pipeline on an unseen clip with the network disabled. This is the only way to prove nothing silently reaches for a download.	It runs offline. If it doesn't, you have found the bug that would have ended your morning.

8:15–9:00 AM	Pack, travel, arrive Laptop, charger, mouse, HDMI adapter, phone hotspot as backup. Weights and datasets already local.	You can run the whole demo with the venue wifi switched off.
The rule for Saturday morning	Retrieval, not revision – and definitely not new material	

RUNBOOK 09:00 to 09:30 Level 1, from a cold start		EXPECTED SHAPE The three levels Inferred from their scope list, not quoted	
0–10m	Sit through the State-of-the-Art session. Read the real level definitions and re-plan if they differ from the guess below.	L1	Detect and score. An anomaly score, a threshold, an evaluation number. Everyone clears this — budget it tight.
10–12m	Point <code>config.yaml</code> at their video. Watch 60 seconds at speed — day or night, crowded or sparse, hovering or moving.	L2	Context and identity. Tracking, dwell, zones, re-ID. Anomalies that exist only across time. Where the field thins.
12–17m	<code>python fit.py --video theirs.mp4</code> → speed and density baselines, normal bank, retrieval index.	L3	Retrieval and forensics. Ask the footage a question in words, get the clip and the reasoning back.
17–20m	Draw two or three zone polygons on a still. One minute of work, disproportionate payoff.	note	The arena says "three levels" without defining them. Confirm in the first ten minutes and re-plan rather than assuming.
20–30m	<code>python run.py --preset day</code> → first alerts, first FPS, first eval table. Level 1 cleared.		
30m+	Climb. Re-ID for level 2, retrieval and forensics for level 3, upgrades after that.		

THE SINGLE BIGGEST ARCHITECTURAL CORRECTION

An anomaly has no class label

Their scope list leads with **open-world** video understanding, and that word decides the detector. A COCO-trained YOLO knows eighty things. An anomaly is, almost by definition, *a thing you had no class for* — so a closed-set detector is structurally the wrong instrument for the headline problem, however well tuned. Even your overnight VisDrone weights are closed-set: better at *seeing small things*, not at naming new ones.

YOLO-World closes that gap by taking text as the class list at inference time. Ask for "person climbing a fence", "abandoned bag", "vehicle on a footpath", with nothing trained. Run both: closed-set for the fast always-on pass, open-vocabulary fired on candidate windows only. Same cascade logic, one layer up.

DOMAIN RISK · THE DIFFERENTIATOR

Night flights break the naive pipeline

They tell you the footage includes night flights. Most entries will tune on daylight, demo on daylight, and fall over when a judge scrubs to the night segment. Handling it on purpose is the cheapest way to look like the only serious submission in the room — and with the overnight run trained on synthetic dark frames, you have a head start nobody else bought.

01 · DETECTOR CONFIDENCE COLLAPSES

Low contrast plus sensor noise drops YOLO scores across the board. CLAHE before inference, and a *separate, lower* `conf` for night. One threshold for both is not robustness.

02 · COLOUR STOPS MEANING ANYTHING

Re-ID leaning on colour histograms fails silently on near-monochrome video. OSNet's learned embedding degrades more gracefully — but say plainly that identity matching is weaker after dark rather than reporting one confidence for both.

03 · THE VLM SEES GRAIN TOO

Qwen's judgement degrades on noisy dark frames just as the detector's does. Measure it: same event set, day and night, both numbers reported. A stated degradation beats an average that hides it.

04 · THE CAMERA MOVES, AND ALTITUDE CHANGES SCALE

Pixel speed conflates object motion with platform motion, and a 45 px dwell radius means different things at 20 m and 80 m. Gate on it — trust speed only while hovering — or stabilise with sparse optical flow. **Naming this limitation scores better than pretending it isn't there.**

FOUNDATIONS · THE LAYER EVERYTHING ELSE SITS ON

The PyTorch you will actually touch

You already know Transformers, PEFT and quantization, so a general PyTorch course would burn hours teaching you what you have. **What you will actually touch is much narrower** — and it is exactly where a five-model pipeline breaks at 2am. Read this list, not a course.

WHAT	WHY IT MATTERS IN THIS PIPELINE	WHERE	MIN
The entry point	If any of the rows below feel unfamiliar, start here and run the code rather than reading it. Otherwise skip straight to the CUDA row	"Learn the Basics" series	—
Tensors: dtype, shape, device	Every model call. Most of your errors will be a shape or a dtype, not logic	Tensors tutorial	15
CUDA semantics — <code>.to(device)</code> , memory ★	Five models moving between CPU and GPU. "Expected all tensors on the same device" is the error you will see most	CUDA semantics	20
<code>torch.inference_mode()</code> / <code>no_grad()</code> ★	Forgetting it builds an autograd graph you never use — the single most	inference_mode	10

	common cause of an OOM that makes no sense		
Half precision — fp16 / bf16, autocast	Half the memory, most of the speed. This is how Qwen and the detector both fit at once	torch.amp	15
4-bit loading (bitsandbytes) ★	Qwen2.5-VL-3B drops from ~7 GB to ~2.5 GB. You know quantization — this is just the BitsAndBytesConfig call	HF quantization	15
<code>.eval()</code> and why it matters	BatchNorm and dropout behave differently. Forget it and your embeddings drift between runs	nn.Module docs	5
Saving / loading <code>state_dict</code>	Swapping in a VisDrone-pretrained checkpoint, and rescuing <code>last.pt</code> if you stop training early	Saving & loading models	10
Debugging CUDA OOM ★	You will hit it. Batch size, <code>max_pixels</code> , <code>empty_cache()</code> , and knowing which of the three to reach for	PyTorch FAQ	15
Starred rows only	≈ 1 h 05 m — read it during T0 and T1 while things install		

THE FOUR CONVERSION TRAPS — THESE ARE LIVE IN YOUR SCAFFOLD RIGHT NOW

BGR vs RGB. OpenCV reads BGR; every model expects RGB. Get it wrong and nothing crashes — detection quality just quietly drops and you spend an hour blaming the weights. `fit.py` uses `f[:, :, ::-1]` and `vlm_judge.py` uses `cv2.cvtColor`; both are doing the same conversion and both must stay.

HWC vs CHW. NumPy and PIL are height-width-channel; PyTorch is channel-first. **uint8 vs float.** A 0–255 array fed to a model expecting 0–1 gives garbage, silently. **The batch dimension.** `unsqueeze(0)` before a single-image call, or you get a shape error two frames deep in a stack trace.

None of these raise an exception. That is what makes them expensive — and it is why block T1's deliverable is "the pipeline runs on one clip", not "the code compiles".

LIBRARIES

Study two, skim the rest

Eight libraries appear in this plan, which reads like eight things to learn. It is really two — the rest is about thirty lines of code between them. The trap is treating each import as a subject.

LIBRARY	WHAT YOU ACTUALLY WRITE	WHERE TO LEARN IT	S
Ultralytics ★	<i>It is</i> your pipeline — detect, track, train, val, export, all one API. Every knob you will be questioned on lives here	docs.ultralytics.com — predict , track , train , the full arg reference	[
PyTorch ★	The narrow list above: <code>device</code> , <code>inference_mode</code> , <code>autocast</code> , 4-bit, OOM. Where a five-model stack breaks at 2am	" Learn the Basics " series — skim and run the code; then the eight rows above	[
OpenCV	Six functions. <code>VideoCapture</code> / <code>read</code> / <code>set</code> , <code>cvtColor</code> , <code>createCLAHE</code> , <code>imread</code> / <code>imwrite</code> , <code>GaussianBlur</code>	video capture · colour spaces (the BGR trap) · CLAHE · blur	[
scikit-learn	One call: <code>roc_auc_score</code> . Plus <code>precision_recall_fscore_support</code>	roc_auc_score · metrics overview	[
open_clip	Three lines: encode, normalise, dot product	github.com/mlfoundations/open_clip — the README's usage block is the whole lesson	[
torchreid	Three lines: load OSNet, embed crops, cosine	deep-person-reid docs · repo	[
SAHI	One wrapper call around the detector you already loaded	Ultralytics SAHI guide (start here) · github.com/obss/sahi	[
bitsandbytes	One <code>BitsAndBytesConfig</code> object. You know quantization already	HF quantization docs — the bitsandbytes 4-bit section only	[
Transformers	Loading Qwen: <code>AutoProcessor</code> , <code>from_pretrained</code> , <code>generate</code> . You	HF Transformers docs — refresher only if rusty	[

have this		
supervision	Convenience only — zones are already hand-rolled in <code>events.py</code> . Skip it if you are short	supervision.roboflow.com
NumPy	You have it	numpy basics — only for the axis/broadcast rules if a shape error stumps you
Ultralytics + PyTorch is two-thirds of your study time Everything below them is API surface, not a subject		

HOW TO USE THE "IN-BLOCK" LINKS

Don't read them tonight. Open the tab *when the block that needs it starts* — OpenCV's colour-space page during T9, torchreid during F1, sklearn during F5. Reading an API you won't touch for six hours is time you don't get back, and you will have forgotten it by then anyway.

CURRICULUM · MAPPED TO THE BLOCK THAT NEEDS IT

What to read, and when

This sits *on top of* the PyTorch hour above — call it 3 h 45 m of real reading across two days. Read the starred rows properly; skim everything else *inside* its block, page open beside the code. Where a video row shows a `search term`, the URL could not be verified, and a link that 404s at hour one is worse than a search box.

CONCEPT	RESOURCE	TYPE	BLOCK	MIN
Fine-tuning YOLO ★	docs.ultralytics.com/modes/train + the full arg reference for augmentation keys	DOC	T3	25
VisDrone as an Ultralytics dataset	docs.ultralytics.com/datasets/detect/visdrone — auto-downloads, one YAML	DOC	T2	10

Frame sampling, FPS, every predict arg ★	docs.ultralytics.com/modes/predict — <code>vid_stride</code> , <code>stream=True</code> , the args table	DOC	T1	25
Confidence vs IoU vs NMS	"IoU and non-max suppression explained" — DeepLearning.AI clip, ~10 min	VIDEO	T1	10
Tracking, track IDs, thresholds ★	docs.ultralytics.com/modes/track plus the tracker YAMLS it links	DOC	T4	20
ByteTrack — <i>why</i> it works	arxiv.org/abs/2110.06864 — abstract and Fig. 2 only	PAPER	T4	10
BoT-SORT — re-ID inside the tracker	arxiv.org/abs/2206.14651 — abstract only	PAPER	T4	5
Small aerial objects	SAHI tiled inference + github.com/obss/sahi	DOC	T4	20
Zones, line-crossing, annotators, traces	supervision.roboflow.com — <code>PolygonZone</code> , <code>TraceAnnotator</code> , <code>DetectionsSmoother</code>	DOC	T5	25
Video anomaly detection — the framing	arxiv.org/abs/1801.04264 (Sultani, UCF-Crime) — intro and problem setup	PAPER	T5	20
Small VLM — the one we use ★	github.com/QwenLM/Qwen2.5-VL → the <code>cookbooks/</code> video notebook and the <code>max_pixels</code> note	REPO	T6	40
VLM architecture — how it actually works ★	Search YouTube: Umar Jamil "Vision Language Models" and Umar Jamil "CLIP" — code-along walkthroughs of the encoder → projection → decoder path	VIDEO	T6	45
CLIP — why	arxiv.org/abs/2103.00020 — abstract and intro	PAPER	T8	15

zero-shot works at all	only. Contrastive training is why open-vocab detection and your normal bank both function			
Practical small-VLM patterns	Search: "huggingface smol-vision" — notebooks matching the exact model sizes you are running	REPO	T6	20
Zero-shot anomaly detection with CLIP	Search: WinCLIP and AnomalyCLIP — the published version of what your normal bank does. Read one abstract; borrow the framing for Q&A	PAPER	F2	20
Live-monitoring metrics	Search: NVIDIA AI City Challenge Track 4 — skim the metric definitions, specifically F1 paired with time-to-detection	PAPER	F5	15
Small VLM — the fallback	huggingface.co/blog/smolvlm2 — tiny, native video	BLOG	T6	20
VLM-based VAD — copy this shape ★	"Harnessing Large Language Models for Training-free Video Anomaly Detection" (LAVAD, CVPR 2024) — the method figure	PAPER	T6	25
Open-vocabulary detection ★	docs.ultralytics.com/models/yolo-world — text prompts as the class list	DOC	T8	30
Grounding DINO — the fallback	github.com/IDEA-Research/GroundingDINO — README and the demo call	REPO	T8	20
Open-world detection — the survey	arxiv.org/pdf/2508.16527 — skim for vocabulary you can use in Q&A	PAPER	T8	15
Low light / night ★	OpenCV histogram equalisation & CLAHE — short, directly usable	DOC	T9	15
Re-identification	github.com/KaiyangZhou/deep-person-reid — OSNet, and the embed-then-cosine pattern	REPO	F1	25
NVIDIA pretrained surveillance	NVIDIA TAO model zoo — verify ReIdentificationNet and PeopleNet names, sizes, licences	DOC	F1	20

models				
Retrieval — text query over video	open_clip README — encode, normalise, dot product	DOC	F2	25
Real-time export	Ultralytics TensorRT export — you know ONNX, so ten minutes	DOC	F3	10
Evaluation for VAD	scikit-learn model evaluation — roc_auc_score, the PR trade-off	DOC	F5	20
SAM 2 — masking that tracks itself	github.com/facebookresearch/sam2 — only if you reach F7	REPO	F7	15
Temporal reasoning	No short resource exists. Learn it in T6: same clip at 1 frame, 6 frames, then 6 frames plus track history in the prompt. Compare verdicts.	HANDS-ON	T6	25
Starred rows only	≈ 2 h 40 m of real reading across two days — the rest is skimmed in- block			

PARAMETERS

The knobs you will be asked about

A mentor's first question is never "what model?" — it is "what happens if you turn this up?"

STAGE	PARAMETER	WHICH WAY, AND WHY
Training	--preset	overnight if the GPU is yours all night; kaggle if it is borrowed; fast if T3 slipped past midnight
Training	freeze	0 overnight — you have the hours for a full fine-tune. freeze=10 only when time-boxed
Training	hsv_v	↑ 0.65. Brightness jitter is cheap night robustness on top of the synthetic dark frames

Training	<code>mosaic</code> / <code>close_mosaic</code>	Mosaic helps small objects a lot; off for the last 8 epochs so the model sees real layouts
Training	<code>night</code> <code>--fraction</code>	0.4. Too high and it forgets daylight; too low and night never registers
Sampling	<code>target_fps</code>	↓ 30→3 is 10× cheaper and loses almost nothing for loitering. Raise only for falls and collisions
Sampling	<code>clip_seconds</code> , <code>overlap</code>	The window must be longer than the event you claim to detect
YOLO	<code>weights</code>	Stock vs your fine-tune. Decided by measured mAP in F0, not by sentiment
YOLO	<code>imgsz</code>	↑ 1280 for aerial . A top-down person is 15 px after a 640 resize, and vanishes
YOLO	<code>conf</code>	↓ 0.20–0.25 by day, lower again at night . Recall here; precision later in the event layer
YOLO	<code>iou</code> , <code>classes</code>	NMS overlap — lower in crowds. Restrict classes: fewer classes, fewer nonsense tracks
YOLO-World	the prompt list itself	Your real tuning surface on the day. "person climbing fence" beats "person". Short noun phrases beat sentences
Night	CLAHE clip limit, tile grid	↑ More clip limit means more contrast <i>and</i> more amplified noise. Find the sweet spot and show it
SAHI	slice size / overlap	640 slices at 0.2 overlap. Big recall jump on drone footage, big FPS cost
Tracker	<code>track_high_thresh</code> / <code>low_thresh</code>	The two-stage association at the core of ByteTrack
Tracker	<code>new_track_thresh</code>	↑ Higher means fewer ghost tracks
Tracker	<code>track_buffer</code>	↑ Frames a track survives occlusion. Raise it when IDs churn

9/3/26, 11:04 PM

Ten-Hour Flight Plan

Tracker	botsort vs bytetrack	botsort adds re-ID: better through occlusion, slower. For level 2 it is worth the cost
Re-ID	cosine match threshold	Too low and two strangers merge; too high and one person becomes three. Report both failure directions
Events	<code>loiter_seconds</code> , <code>loiter_radius_px</code>	The radius is in pixels , so it is altitude-dependent on a drone. Say that out loud
Events	<code>speed_z_threshold</code>	z against the baseline <code>fit.py</code> measured on <i>their</i> video, never a global constant
Events	<code>min_track_frames</code>	Refuse to judge blips
VLM	<code>max_pixels</code>	↓ Halve it, roughly halve the latency. Your number-one lever
VLM	<code>frames_per_event</code>	4–8. Below 4 you lose temporality; above 8 you pay for nothing
VLM	<code>temperature = 0</code>	A judge that changes its mind between runs is unusable
Fusion	<code>w_geometric</code> / <code>w_vlm</code>	Geometry is measured, the VLM is opinion. Don't let opinion dominate
Fusion	<code>raise</code> / <code>clear</code>	Hysteresis — raise high, clear low. This is what stops alert flicker
Fusion	<code>cooldown_seconds</code>	Per track ID. Stops forty alerts for one loiterer

DATA

They supply footage — bring labels anyway

The arena provides real urban drone footage and benchmark datasets. That covers your demo. It does not necessarily cover your *metrics*, and a frame-labelled set is what turns a demo into a number.

DATASET	ROLE IN THIS PLAN	WHERE
https://claude.ai/code/artifact/02726993-e5f7-49c0-840e-72b258cf6585		

VisDrone ★ start tonight	The training set. Gates block T3, and it is not small. Also your re-ID practice ground	aiskyeye.com · auto-download via Ultralytics
CUHK Avenue ★	Where your AUC comes from. Small, frame-level labels, fast to evaluate	"CUHK Avenue anomaly dataset"
UCF-Crime	128 h of real surveillance, 13 anomaly classes, weak labels. Good for the forensic demo in F6	"UCF-Crime dataset"
ShanghaiTech Campus	Multi-scene, frame-level labels, the VAD standard. A second AUC if F5 has time	"ShanghaiTech Campus anomaly dataset"
UBnormal	Synthetic, pixel-level, with <i>unseen</i> anomaly types at test — a real generalisation check	"UBnormal CVPR 2022 github"
Drone-Anomaly	Small aerial VAD set — closest published match to the actual task	"Anomaly Detection in Aerial Videos with Transformers"
Two are mandatory VisDrone gates the training · Avenue gates the metrics		

THE LEAKAGE QUESTION YOU WILL BE ASKED

Split by video, never by frame. Adjacent frames are near-duplicates, so frame-level shuffling hands you a fake 0.99 AUC — and a panel that stops trusting the rest of your numbers. Same applies to the synthetic night images: `x.jpg` and `x_night.jpg` must land on the same side of the split.

EVALUATION

The numbers that go on the slide

